B 9 I S 1 3 0

Web Development for Information Systems

Continuous Assessment 1 — Project Briefing

MSc Computing with Information Systems
Dublin Business School
Module Leader: Dr. Obinna Izima

Assessment 1 of 2 · 85% Module Weighting · Individual Project

Assessment at a Glance

Three components, one integrated project

70	20	10
MARKS	MARKS	MARKS
Web Application	**Technical Report**	**Presentation & Demo**
Part 1	**Part 2**	**Part 3**

ISSUE DATE	DUE DATE	FEEDBACK	MODE	SUBMISSION
30/09/2026	10/11/2026, 23:59	Hand-in + 2 weeks	Individual	Online via Moodle

Minimum 2,000 words for the Technical Report · Restrictions apply as per the module brief

What You Are Building

A complete, modern full-stack web application or proof-of-concept information system

1

Requirements Analysis

Problem domain, users, functional & non-functional requirements

2

Front-End & UX

Responsive, accessible, professional interface

3

Back-End & Database

Secure, modular server-side logic with CRUD

4

API Integration

Consuming/exposing web services, JSON/XML exchange

5

Security

AuthN/AuthZ, validation, OWASP-aware mitigations

6

Software Engineering

Version control, modularity, design patterns

7

Cloud Deployment

Publicly accessible, production-style hosting

Plus: a well-maintained GitHub repository throughout development.

The JavaScript Full-Stack Requirement

MERN or MEAN stack is strongly preferred

Projects built without a JavaScript-based implementation will be considered fundamentally limited in scope and may be deemed flawed in meeting the module's intended learning outcomes.

What this means in practice

- ✓ Front end and back end both implemented in JavaScript/TypeScript
- ✓ React, Angular, or Vue on the front end
- ✓ Node.js + Express (or NestJS) on the back end
- ✓ MongoDB is the natural database fit for MERN/MEAN

Recommended Tech Stacks: MERN vs MEAN

MERN

- ▸ MongoDB — NoSQL document DB
- ▸ Express.js — REST API framework
- ▸ React — component-based UI
- ▸ Node.js — JS runtime

Best for: dynamic SPAs, dashboards, highly interactive UIs

MEAN

- ▸ MongoDB — NoSQL document DB
- ▸ Express.js — REST API framework
- ▸ Angular — opinionated full framework
- ▸ Node.js — JS runtime

Best for: larger structured apps, teams wanting built-in conventions

Other JS-based options (with justification): Next.js full-stack, Vue + Nuxt, NestJS + PostgreSQL/Prisma, Firebase for rapid prototyping.

1 Requirements Analysis & Planning

15% of Part 1 — the foundation examiners check first

HOW TO SUCCEED

- ✓ Define a clear problem domain and named target users/stakeholders
- ✓ Write explicit functional AND non-functional requirements
- ✓ Show evidence of research (market gap, similar tools, feasibility)
- ✓ Use a simple use-case diagram or user story set
- ✓ Scope the project to what is achievable in the timeframe

COMMON PITFALLS

- ✗ Jumping into code with no documented requirements
- ✗ Copy-pasted generic requirements not tied to your idea
- ✗ No mention of non-functional requirements (performance, security, usability)
- ✗ Scope creep: an idea far too large for a solo project
- ✗ No stakeholder/persona thinking — "just a to-do app"

2 Front-End Development & UX Design

Judged on usability, consistency, accessibility, and polish

HOW TO SUCCEED

- ✓ Use a component-based framework (React, Angular, Vue) consistently
- ✓ Design for responsiveness — test mobile, tablet, desktop breakpoints
- ✓ Apply accessibility basics: alt text, contrast, semantic HTML, labels
- ✓ Keep navigation predictable; give clear feedback for every action
- ✓ Use a consistent design system (spacing, colour, typography)

COMMON PITFALLS

- ✗ Inconsistent styling across pages/components
- ✗ Ignoring mobile responsiveness entirely
- ✗ Unlabelled forms, poor colour contrast, no keyboard navigation
- ✗ Cluttered layouts with unclear calls to action
- ✗ Using raw unstyled HTML and calling it "done"

3 Back-End Development & Database Integration

Secure, modular, and demonstrably CRUD-complete

HOW TO SUCCEED

- ✓ Structure the API in layers: routes, controllers, services, models
- ✓ Implement full CRUD with proper HTTP status codes
- ✓ Validate all incoming data server-side, not just client-side
- ✓ Design a sensible schema (MongoDB collections / SQL tables + relations)
- ✓ Add centralised error handling and meaningful error messages

COMMON PITFALLS

- ✗ Business logic dumped directly into route handlers
- ✗ Trusting client-side validation alone
- ✗ No error handling — the app crashes on bad input
- ✗ Hard-coded database credentials committed to GitHub
- ✗ Read-only apps with no real Create/Update/Delete flows

4 API & Web Service Integration

Demonstrate real client-server data exchange

HOW TO SUCCEED

- ✓ Integrate at least one meaningful external API (weather, maps, payments, etc.) OR expose a clean internal REST API
- ✓ Use JSON consistently; document your endpoints (README or Swagger)
- ✓ Handle async calls properly: loading, success, and error states
- ✓ Use environment variables for API keys — never hard-code them

COMMON PITFALLS

- ✗ Fake or decorative "API calls" that do nothing meaningful
- ✗ No loading or error states — UI breaks silently on failure
- ✗ Committing API keys/secrets to the public repository
- ✗ Confusing REST conventions (wrong verbs/status codes)

5 Security Implementation

Aligned with OWASP Top 10 awareness

HOW TO SUCCEED

- ✓ Hash passwords (bcrypt/argon2) — never store plaintext
- ✓ Implement authentication and role-based authorisation (JWT/sessions)
- ✓ Validate and sanitise all inputs to prevent SQLi / NoSQLi and XSS
- ✓ Use HTTPS, secure cookies, and environment-based secrets
- ✓ Protect routes: check auth/roles on the server, not just hide UI

COMMON PITFALLS

- ✗ Storing plaintext passwords or weak/no hashing
- ✗ Client-side-only authorisation (hidden button ≠ protected route)
- ✗ No input sanitisation — vulnerable to injection and XSS
- ✗ Secrets committed to Git history
- ✗ "We didn’t have time for security" — this criterion is heavily weighted

6 Software Engineering Practice & Code Quality

Maintainability and process are marked, not just the demo

HOW TO SUCCEED

- ✓ Commit early and often with meaningful, descriptive messages
- ✓ Organise code into modules/folders by feature or layer
- ✓ Apply a recognisable design pattern (MVC, service layer, repository)
- ✓ Use linting/formatting (ESLint, Prettier) for consistency
- ✓ Write a clear README: setup, run, architecture overview

COMMON PITFALLS

- ✗ One giant commit the night before the deadline
- ✗ Everything in a single file ("index.js" doing everything)
- ✗ No comments, no README, no explanation of structure
- ✗ Dead code, console.logs, and commented-out blocks left in
- ✗ Copy-pasted boilerplate the student cannot explain

7 Cloud Deployment

The application must be live and publicly reachable

HOW TO SUCCEED

- ✓ Deploy front end and back end (e.g. Vercel/Netlify + Render/Railway)
- ✓ Use MongoDB Atlas (free tier) for a cloud-hosted database
- ✓ Configure environment variables properly on the host, not hard-coded
- ✓ Test the live link yourself, from a different network, before submitting
- ✓ Keep the deployment stable for the whole marking window

COMMON PITFALLS

- ✗ Only ever demoing from localhost
- ✗ Deployed link that returns errors or has expired free-tier limits
- ✗ Forgetting to set environment variables on the cloud platform
- ✗ Database not reachable from the deployed back end (IP allow-list)
- ✗ Submitting the link at 23:58 without testing it first

Cloud Deployment Options

Free-tier friendly platforms suitable for a postgraduate coursework project

Front End (static/SPA)

- Vercel
- Netlify
- GitHub Pages
- Render (static site)

Back End (Node/Express API)

- Render
- Railway
- Fly.io
- Azure App Service

Database

- MongoDB Atlas (free M0 cluster)
- ElephantSQL (Postgres, if SQL used)
- Supabase (Postgres + auth)

Part 2 — Technical Report (20 Marks)

Minimum 2,000 words, Harvard referencing throughout

SUGGESTED STRUCTURE

1. Introduction & problem statement
2. Requirements & research process
3. System architecture & technology justification
4. Security implementation & threat mitigation
5. Testing strategy & results
6. Deployment process & configuration
7. Critical reflection & limitations
8. Harvard-style reference list

WHAT LOSES MARKS

- ✗ Description instead of critical discussion ("what" not "why")
- ✗ No in-text citations, or reference list not in Harvard style
- ✗ Report doesn't match what was actually built/deployed
- ✗ Missing testing evidence or deployment walkthrough
- ✗ Padding word count without adding substance

Part 3 — Presentation & Demonstration (10 Marks)

10–15 minutes, delivered in class

1

Problem & Users

1-2 min: what problem, for whom, why it matters

→

2

Architecture

2-3 min: stack, structure, key design decisions

→

3

Live Demo

5-7 min: core workflows, including security & error handling

→

4

Reflection & Q&A

2-3 min: challenges, what you'd improve, answer questions

Tip: rehearse the demo path end-to-end beforehand, and have a backup (screen recording) in case of Wi-Fi issues.

Suggested Project Timeline

Issue 30 September 2026 → Due 10 November 2026 (approx. 6 weeks)

Week 1	Week 2	Week 3	Week 4	Week 5	Week 6
Requirements, research, wireframes, repo + project setup	Backend: schema, models, core CRUD API, auth scaffolding	Frontend: core screens wired to API, basic styling	Security hardening, API integration, validation, testing	Polish UX, deploy to cloud, fix deployment issues	Write technical report, rehearse demo, final QA & submit

Do not attempt cloud deployment for the first time in the final 48 hours.

Common Pitfalls That Cost Marks

- Choosing a non-JavaScript stack against the brief's clear guidance
- Over-ambitious scope that leaves security/deployment unfinished
- No real database — hard-coded/mock data pretending to be dynamic
- Weak or absent authentication and authorisation
- Thin GitHub history (one commit, no branches, no messages)
- Report that doesn't match the actual submitted application
- Skipping accessibility and responsive design entirely
- Deploying at the last minute without testing the live URL
- Heavy unacknowledged use of AI-generated code (academic integrity)
- Not rehearsing the demo — technical issues eat into presentation time

How to Reach High Distinction

What separates a Pass from a High Distinction (70%+), per the rubric

CRITERION	PASS (40–49%) LOOKS LIKE	HIGH DISTINCTION (70%+) LOOKS LIKE
Requirements	Basic problem statement	Comprehensive, justified, stakeholder-aware analysis
Front-End/UX	Functional but inconsistent	Professional-quality, accessible, polished interaction design
Back-End/DB	CRUD partially working	Robust, scalable, secure data architecture
Security	Basic measures only	OWASP-aligned, comprehensive threat mitigation
Deployment	Partial / unstable	Production-ready, reliable, professionally configured

Submission Checklist

All four items are required via Moodle — missing links can cost marks

1	**Technical Report**	PDF or Word format, 2,000+ words, Harvard referenced
2	**Presentation**	Delivered in class, 10-15 minutes
3	**GitHub Repository URL**	Public/accessible, meaningful commit history
4	**Deployed Application URL**	Live, publicly reachable, tested before submission

Academic Integrity: all work must be your own. Any use of external code, AI-generated content, or third-party resources must be clearly acknowledged and understood by you.

Sample Project Ideas

Starting points — refine scope and add your own domain twist

Campus Lost & Found Portal

Report/search lost items, image upload, claim workflow, admin moderation

Student Micro-Freelance Marketplace

Post gigs, apply, messaging, ratings, secure payments (mock)

Clinic/Appointment Booking System

Slot booking, role-based access (patient/staff), reminders via API

Personal Finance & Budget Tracker

Expense categorisation, charts, bank-style API integration

Community Volunteering Platform

Event listings, sign-ups, geolocation via maps API

Peer Learning / Study Group Finder

Match by subject, real-time chat, calendar API integration

Questions?

Office hours, Moodle forum, and module leader email are the fastest routes for clarifications on scope or grading.